

Source: <http://www.faadooengineers.com/online-study/post/cse/database-management-systems/352/granting-of-locks>

- In lock based protocol A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions.
- Any number of transactions can hold shared locks on an item.
- If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transactions have been released. The lock is then granted.

Granting of Locks

- When a transaction requests a lock on a data item in a particular mode, and no other transaction has a lock on the same data item in a conflicting mode, the lock can be granted
- Care must be taken to avoid the following scenario
 - Suppose a transaction T2 has a shared-mode lock on a data item, and another transaction T1 requests an exclusive-mode lock on the data item.
 - Clearly, T1 has to wait for T2 to release the shared-mode lock.
 - Meanwhile, a transaction T3 may request a shared-mode lock on the same data item.
 - The lock request is compatible with the lock granted to T2, so T3 may be granted the shared-mode lock. At this point T2 may release the lock, but still T1 has to wait for T3 to finish.
 - But again, there may be a new transaction T4 that requests a shared-mode lock on the same data item, and is granted the lock before T3 releases it.
 - In fact, it is possible that there is a sequence of transactions that each requests a shared-mode lock on the data item, and each transaction releases the lock a short while after it is granted, but T1 never gets the exclusive-mode lock on the data item.
 - The transaction T1 may never make progress, and is said to be starved.
- We can avoid starvation of transactions by granting locks in the following manner:
When a transaction T_i requests a lock on a data item Q in a particular mode M , the concurrency-control manager grants the lock provided that

1. There is no other transaction holding a lock on Q in a mode that conflicts with M.
2. There is no other transaction that is waiting for a lock on Q, and that made its lock request before T_i . Thus, a lock request will never get blocked by a lock request that is made later.